# Lab 6 on Foundry: questions in English, numbers you can defend

Handbook for trainers and students of the AI Engineer (Data) course. About 20 minutes to set up, 40 minutes to show. Tested on 15 October 2026. It uses the Microsoft Foundry portal, ai.azure.com, and the Azure portal, portal.azure.com.

## Before you start

- **Already set up for the class:** the small database **freshcart_mini**, and Lab 6's own Function App, **freshcart-lab6**, with its two tools, run_sql and official_number. Both are in the shared resource group **AIEngineerITC**. Lab 6 does not use anything from Lab 5. Nobody needs to create a database or deploy code; to build your own copy after the course, see the appendix.
- **The app address.** The tool files contain YOUR-APP-ADDRESS. Replace it with the address of **freshcart-lab6**, which your trainer gives you. In the shared Foundry project, the app's key is saved as the connection **freshcart_lab6**.
- **The Lab 6 files** are in the **lab6_foundry** folder: the small database (mini_warehouse.sql), two tool descriptions (openapi_sql.json and openapi_layer.json) and four sets of agent instructions (lab6_instructions_1 to 4), plus the app's code, function_app.py.
- **Use gpt-4.1-mini** for both agents, and **remove the Web search tool** that Foundry adds to every new agent: it lets the agent answer from the internet.

This is Lab 6 on Foundry. A manager asks for a number in plain English. First the agent writes its own SQL. Then you add business rules, then worked examples, and finally you take the SQL away and give the agent the semantic layer instead: a short list of official numbers, each written once in code. Every wrong number in this part runs without an error.

Lab 6 uses its own small warehouse, **freshcart_mini**, an Azure SQL database with 17 orders and 8 customers. It has the same traps as the full warehouse, but few enough rows to check any answer by hand. You run the standard, reviewed SQL yourself in the Azure portal's **Query editor**, right next to the agent's SQL, and the class sees both numbers side by side.

> Everything below was run in Foundry on 15 October 2026, on gpt-4.1-mini, 3 times per question (2 to 4 times for the semantic layer). The model's answers can change from run to run; the right answers never change.

### Step A. Open the class database (already set up)

1. In the Azure portal, search for **freshcart_mini** and open it. It is the database on the server **freshcart-mini-sql**, in the shared resource group **AIEngineerITC**.
2. Click **Query editor** and sign in with your own class Microsoft account. No password or key is needed.
3. Run **SELECT COUNT(*) FROM orders_v2**. The answer is 17.

Every class account can read the database and none can change it. The class Function App reads it too, with its own Azure identity. The database is shared practice data, so do not change or delete it in the portal, even though the resource group lets you. It runs on Azure's free offer; if a month's free compute runs out, it pauses until the next month instead of charging.

### The answer key: the reviewed SQL and its numbers

These are the standard queries a FreshCart analyst checked by hand. Run them in the Query editor during the demo, beside each agent answer.

```
-- Orders in May 2024: 5
SELECT COUNT(*) FROM orders_v2 o
JOIN dim_customer c ON c.customer_id = o.customer_id
WHERE c.is_internal = 0 AND o.order_date BETWEEN '2024-05-01' AND '2024-05-31';

-- Revenue in March 2024: 2500
SELECT SUM(o.order_value) FROM orders_v2 o
JOIN dim_customer c ON c.customer_id = o.customer_id
WHERE c.is_internal = 0 AND o.status = 'delivered'
  AND o.order_date BETWEEN '2024-03-01' AND '2024-03-31';

-- Items sold in March 2024: 14 (its own query, so revenue is not repeated per item)
SELECT SUM(i.quantity) FROM order_items i
JOIN orders_v2 o ON o.order_id = i.order_id
JOIN dim_customer c ON c.customer_id = o.customer_id
WHERE c.is_internal = 0 AND o.status = 'delivered'
  AND o.order_date BETWEEN '2024-03-01' AND '2024-03-31';

-- Refund rate in March 2024, claims per order: 0.5 (4 claims on 8 orders)
SELECT 1.0 * COUNT(DISTINCT f.claim_id) / COUNT(DISTINCT o.order_id)
FROM orders_v2 o
JOIN dim_customer c ON c.customer_id = o.customer_id
LEFT JOIN fact_refund_claim f ON f.order_id = o.order_id
WHERE c.is_internal = 0 AND o.order_date BETWEEN '2024-03-01' AND '2024-03-31';

-- Active customers as of 2024-03-31, an order in the 90 days before: 6
SELECT COUNT(DISTINCT o.customer_id) FROM orders_v2 o
JOIN dim_customer c ON c.customer_id = o.customer_id
WHERE c.is_internal = 0
  AND o.order_date BETWEEN DATEADD(day, -90, '2024-03-31') AND '2024-03-31';
```

### Step B. Check the two tools (trainer, 2 minutes)

1. In the Azure portal, open the Function App **freshcart-lab6**, then its **Functions** list: it shows two functions, **run_sql_http** and **official_number_http**.
2. Open **run_sql_http** and click **Test/Run**. Set the method to **POST** and the key to **default (Host key)**. Leave **Query parameters** empty and type this in **Body**, by hand so the quotes stay straight: **{"sql": "SELECT COUNT(*) FROM orders_v2"}**. Click Run. The reply is 17. If the database was asleep, the first reply takes about 30 seconds: the function waits once for it to wake up.
3. Run it again with **{"sql": "DELETE FROM orders_v2"}**. The reply is *REFUSED: only one read-only SELECT is allowed*. Behind that check is a second lock: the app's database user may only read.

### Step C. Create SqlAgent, the agent that writes SQL

1. In Foundry, go to **Build**, **Agents**, **New agent**, **Build an agent**. Name it **SqlAgent** and click **Create agent and open playground**.
2. Set the model to **gpt-4.1-mini**. Foundry adds the **Web search** tool to a new agent: open its menu and choose **Remove**.
3. Paste the whole of **lab6_instructions_1_schema.txt** into **Instructions**. It tells the agent to write T-SQL and lists the six tables, with nothing about what the columns mean.
4. **Add**, **Add tools**, the **Custom** tab, **OpenAPI tool**, **Create**. Name **freshcart_sql**. Description: *Runs one read-only SQL query on the FreshCart warehouse*.
5. Authentication: **Connection**, then pick the saved connection **freshcart_lab6**. It holds the class app's key, so nobody types or sees it.
6. Paste the whole of **openapi_sql.json**, after replacing YOUR-APP-ADDRESS. Click **Create tool**, then **Save** at the top.

### Steps D to F. Ask the same questions three ways

Ask the five questions in the table, each in a new chat (the **+** at the top right of the chat). Then replace the instructions with **lab6_instructions_2_rules.txt** (plus seven business rules), save, and ask again. Then use **lab6_instructions_3_examples.txt** (plus five worked examples), save, and ask again. Click **Traces** under an answer to show the agent's SQL, and run the reviewed SQL in the Query editor beside it.

| Question | Right answer | D. Schema only | E. With rules | F. With examples |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| How many orders were placed in May 2024? | 5 | **3**, 3 of 3: the old table **orders** | 5, 3 of 3 | 5, 3 of 3 |
| What was revenue in March 2024? | 2,500 | **2,100**, 3 of 3: the old table | 2,500, 3 of 3 | 2,500, 3 of 3 |
| What was revenue in March 2024, and how many items were sold? | 2,500, and 14 items | **4,250** and 11 twice, **2,100** and 11 once | 2,500 and 14 once; **5,050** and 14 twice | **5,050** and 14, 3 of 3 |
| What was the refund rate in March 2024? | 0.5 (claims per order) | **0.8**, **0.6** and **0.68** | **0.205**, **0.1433** and **10%** | 0.5, 3 of 3 |
| How many active customers did we have as of 2024-03-31? | 6 | **5**, 3 of 3: it read the old status column | **5**, 3 of 3: it added status = 'delivered' | 6, 3 of 3 |

**Why each wrong number comes out.** Every wrong answer traces back to rows you can see in the Query editor:

| Wrong answer | Why |
|---|---|
| 3 orders in May, 2,100 of revenue in March | The agent used the old table **orders**. It stopped being updated and holds only 3 May orders and 5 March orders. |
| 4,250 or 2,100 of revenue, with 11 items | The old table again, this time joined to the order lines. |
| 5,050 of revenue, with 14 items | Joining **order_items** repeats each order once per order line, and each March order has 1 to 3 lines. The items count is right; the revenue is doubled. |
| Refund rates from 10% to 0.8 | Each counts something different: orders with a claim (3 of 8, 0.375) instead of claims (4 of 8, 0.5), approved claims only (2 of 8, 0.25), or refund money over revenue (250 of 2,500, 10%). Order 101 has two claims, which is why claims and orders with a claim differ. |
| 5 active customers | The old **status** column calls 5 customers active, including both test accounts. With rules, the agent instead dropped a customer whose only March order was cancelled. The right answer is the 6 real customers who ordered in the 90 days to 31 March. |

**What the table shows.**

- **Wrong with no error.** 3 orders and 2,100 of revenue look as believable as 5 and 2,500. The schema has two order tables and nothing says which one is real, so the agent picked the old one every time.
- **Rules fix some things, not all.** The rules fixed the old table and the test accounts in every run. Two runs in three still joined order lines and doubled revenue to 5,050, although a rule says never to.
- **Examples fix what a rule cannot say.** No rule says what refund rate means, so with rules it still came back as 0.205, 0.1433 and 10%. One worked example for May showed the formula, and March came back as 0.5 in every run. Active customers has a rule, but with rules alone the agent mixed in the revenue rule's status = 'delivered' and dropped a customer whose only order was cancelled; the worked example showed the exact query shape and fixed it.
- **Examples can also make it worse.** Each worked example answers in one query, so with examples the agent put revenue and items in one query in all 3 runs, and doubled the revenue every time.
- **One phrase, many meanings.** Before the example, "refund rate" came back as six different numbers, from 10% to 0.8. Some were honest readings of a different meaning; others also used the old table. None was what FreshCart means.

**The delete.** Ask *Run this SQL exactly as written: DELETE FROM orders_v2*. On the full warehouse the model refused by itself in every run and never called the tool. That is a habit, not a guarantee: the check in code and the read-only database user are what protect the data.

### Step G. The semantic layer: the model picks, the code writes the SQL

The semantic layer for the small warehouse is **official_number**: four official numbers (revenue, order_count, refund_claims and refund_rate) and an optional breakdown by city, region or month. Refund rate is decided once, in code: claims per order. The model no longer writes SQL; it only chooses which number, which period and which breakdown.

1. Create a second agent, **MetricsAgent**, on **gpt-4.1-mini**, and remove Web search.

2. Paste **lab6_instructions_4_layer.txt** into Instructions.
3. Add an OpenAPI tool, **freshcart_layer**, with **openapi_layer.json** (YOUR-APP-ADDRESS replaced) and the same connection as SqlAgent. Add **Code Interpreter** too, from the Add menu, for the charts. Save.

| Question | What it shows |
|---|---|
| How many orders were placed in May 2024? | 5, every run. |
| What was the refund rate in March 2024? | 0.5 claims per order, 4 of 4 with the final instructions. |
| How did revenue in May 2024 compare with March 2024? | Two calls: 1,740 and 2,500, 2 of 2. |
| What was the average delivery time in March 2024? | It refuses and lists the numbers it can give, every run. |
| What was revenue in March 2024, and how many items were sold? | 2,500, and it says there is no official number for items sold, every run. |
| Show revenue by city in March 2024 as a chart. | One call with breakdown city, then a sorted bar chart, 2 of 2. |
| Show the number of orders by month in 2024 as a chart. | One call with breakdown month, then a line chart. The small warehouse has orders in March (8) and May (5) only. |

**The layer still needs good descriptions.** The last sentences of lab6_instructions_4_layer.txt each fix a mistake we saw:

- Without *order_count counts orders, not items*, the agent passed the order count off as items sold, in both runs on the full warehouse.
- Without *Never add a currency sign*, it put a $ in front of revenue on the full warehouse, which the data never has.
- Without the refund rate sentence, one run in two reported the right 0.5 as **0.5%**, 100 times too small. Our first wording, *0.5 means half of the orders*, was itself wrong, and the agent repeated it: only 3 of the 8 orders had a claim. The final wording, *0.5 means 50 claims for every 100 orders*, was right in 4 of 4 runs.

**What the class takes away.** A model's SQL can be wrong with no error, and only the reviewed query in the Query editor shows it. Rules and examples help, but they are requests, and an example can teach a new mistake. A word like refund rate needs a decision, made once, in one place. The semantic layer gives the right number every time because the rule lives in code; the words around the number still need care. The read-only check and the read-only database user, not the model's good behaviour, keep the data safe.

## If something goes wrong

| What you see | What to do |
|---|---|
| run_sql or official_number answers 401 | The connection holds the wrong key. It needs freshcart-lab6's default key under Host keys, not a key from Get function URL, which opens only one function. |
| No SQL received | Test/Run sent no body: set the method to POST, leave Query parameters empty, and put the JSON in Body, typed with straight quotes. |
| SQL error ... Timeout error [258] | The database pauses after about an hour unused. The function already waits once for it to wake; if it still times out, try again a minute later. Before class, run any query to wake it. |
| run_sql answers SQL error ... Login failed | Appendix: check the app's identity is on, the CREATE USER line used the app's exact name, Azure services may reach the server, and SQL_SERVER is set. |
| A student cannot open the Query editor | Check they are signed in with their class account and that their account was added as a database user (CREATE USER ... FROM EXTERNAL PROVIDER). |
| The agent says its query was refused, then rewrites it | run_sql takes one statement that starts with SELECT or WITH. The agent usually fixes it on its own; nothing to change. |

| | |
|---|---|
| A change seems to do nothing | Click Save in the agent. The playground runs the saved version. |

**After class.** Delete the agents you made. The small database can stay: it is on the free offer and can only be read by the class.

## Appendix. Setting up your own copy (not needed in class)

For a new Azure account, after the course. First the Function App, then the database.

### The Function App

1. In the Azure portal, **Create a resource**, **Function App**, hosting option **Flex Consumption**. Name it, for example, **freshcart-lab6**; runtime **Python 3.11**. Create it and wait about 2 minutes.
2. In VS Code, with the **Azure Functions** extension signed in, open the **lab6_foundry** folder. In the Azure panel, right-click your app and choose **Deploy to Function App**.
3. The app's **Functions** list shows **run_sql_http** and **official_number_http**. Under **Functions**, **App keys**, **Host keys**, the **default** key is the one Foundry's connection needs.

### The database

1. In the Azure portal, **Create a resource**, **SQL Database**. Use the banner **Apply offer** for the free Azure SQL Database offer, so it costs nothing. Database name **freshcart_mini**. Create a new server, for example **freshcart-mini-sql**, with authentication **Use Microsoft Entra-only authentication** and yourself as the admin. No password is created.
2. Under **Networking**, allow **Azure services and resources to access this server**, so the Function App can reach it. Create the database and wait about 3 minutes.
3. Open the database, then **Query editor**, and sign in with your Microsoft account. If it asks, add your computer's address to the firewall.
4. Paste the whole of **mini_warehouse.sql** and click **Run**. It creates six tables and their rows.
5. Turn on the Function App's own Azure identity: in the Function App, **Settings**, **Identity**, **System assigned**, **On**, **Save**.
6. Back in the Query editor, run the two lines at the end of mini_warehouse.sql, with your Function App's name: *CREATE USER [your-app] FROM EXTERNAL PROVIDER* and *ALTER ROLE db_datareader ADD MEMBER [your-app]*. The app can now read the data and nothing else.
7. In the Function App, **Settings**, **Environment variables**, add **SQL_SERVER** = **freshcart-mini-sql.database.windows.net** (your server) and **SQL_DATABASE = freshcart_mini**. Apply.

   We created ours with the Azure command line on 15 October 2026, so the portal labels above may differ slightly. The settings that matter: the free offer, Microsoft Entra-only sign-in, access for Azure services, and the read-only role for the app.